



VIT[®]
BHOPAL
www.vitbhopal.ac.in

SCHOOL OF COMPUTING SCIENCE AND ENGINEERING

A PROJECT REPORT ON

Library Management System

B.Tech, Artificial Intelligence and Machine Learning

Submitted by-
Name: Aryan Rusia
Registration No.: 25BAI10193

1. Introduction

This project uses several core Java topics, including fundamentals and flow control, object-oriented programming, exception handling, multithreading, the Collections framework, Java I/O streams, and JDBC database access.

The application developed for this project is a command-line Library Management System. It allows a librarian to manage books and members, issue and return books, renew loans, calculate overdue fines, and export reports. The data is stored in an SQLite database and accessed through JDBC.

The aim was to use the syllabus topics in one working application instead of demonstrating each topic through separate, unrelated examples.

2. Problem Statement

Libraries that use paper registers or basic spreadsheets can have difficulty keeping track of several things:

- Which books are currently available
- Which books have been issued
- When a member must return a book
- How much an overdue fine should be
- Which member borrowed a particular book
- The history of previous loans

This project provides a terminal-based system for managing these tasks in one place. It stores the records in a database, applies the borrowing rules automatically, calculates fines, and records important activities in a log file.

A background process also checks for overdue loans while the main menu is running.

3. Functional Requirements

The system has three main functional areas.

3.1 Catalog Management

The librarian can:

- Add books
- Remove books
- View all books
- Search for books by keyword
- Search by title, author, genre, or ISBN

3.2 Member Management

The librarian can:

- Register members
- View all members
- Search for members
- Register members under the STUDENT, FACULTY, or GUEST categories
- Track each member's active loan count
- Track outstanding fines

Each membership category has its own borrowing limit and loan period.

3.3 Lending Workflow

The system supports the complete lending process:

- Issue a book
- Return a book
- Renew a loan
- Check whether a book is available
- Check whether the member exists
- Check whether the member has reached the borrowing limit
- Check whether a loan can be renewed
- Calculate fines when a book is returned late

Business-rule failures are handled with custom exceptions.

3.4 Supporting Features

The system also includes:

- Catalog report export
- Overdue-loan report export
- A timestamped activity log
- A background thread that checks for overdue loans
- SQLite database storage

Reports are written as .txt files.

4. Non-Functional Requirements

Requirement	Implementation
Performance	Frequently accessed book records are stored in a HashMap cache in the service layer. SQLite queries use primary-key lookups where appropriate.
Reliability	Database operations use parameterised PreparedStatement objects inside try-with-resources blocks. This ensures that connections and statements are released after use.

Requirement	Implementation
Security	SQL statements use parameters instead of string concatenation. This prevents user input from being inserted directly into SQL queries.
Usability	The application uses a numbered menu with simple prompts. The librarian does not need to remember command names or flags.
Maintainability	The project separates the model, DAO, service, UI, thread, and utility packages. Database code, business rules, and user interaction are kept in separate layers.
Error handling	Business-rule failures use a hierarchy of custom checked exceptions based on <code>LibraryException</code> . Invalid input such as a malformed ISBN is handled by <code>InvalidISBNException</code> , which is unchecked.
Logging and monitoring	Issue, return, renewal, and overdue-check events are appended to <code>reports/activity.log</code> with timestamps.
Resource efficiency	A shared JDBC connection is reused for the lifetime of the application instead of opening a new connection for every query.

5. System Architecture

The application uses a layered architecture. The main layers are:

- 1 Presentation
- 2 Business logic
- 3 Data access
- 4 Utilities

The presentation layer contains the command-line interface. The business-logic layer applies the library rules. The data-access layer communicates with SQLite. The utility classes handle smaller tasks such as ISBN validation, fine calculation, and report generation.

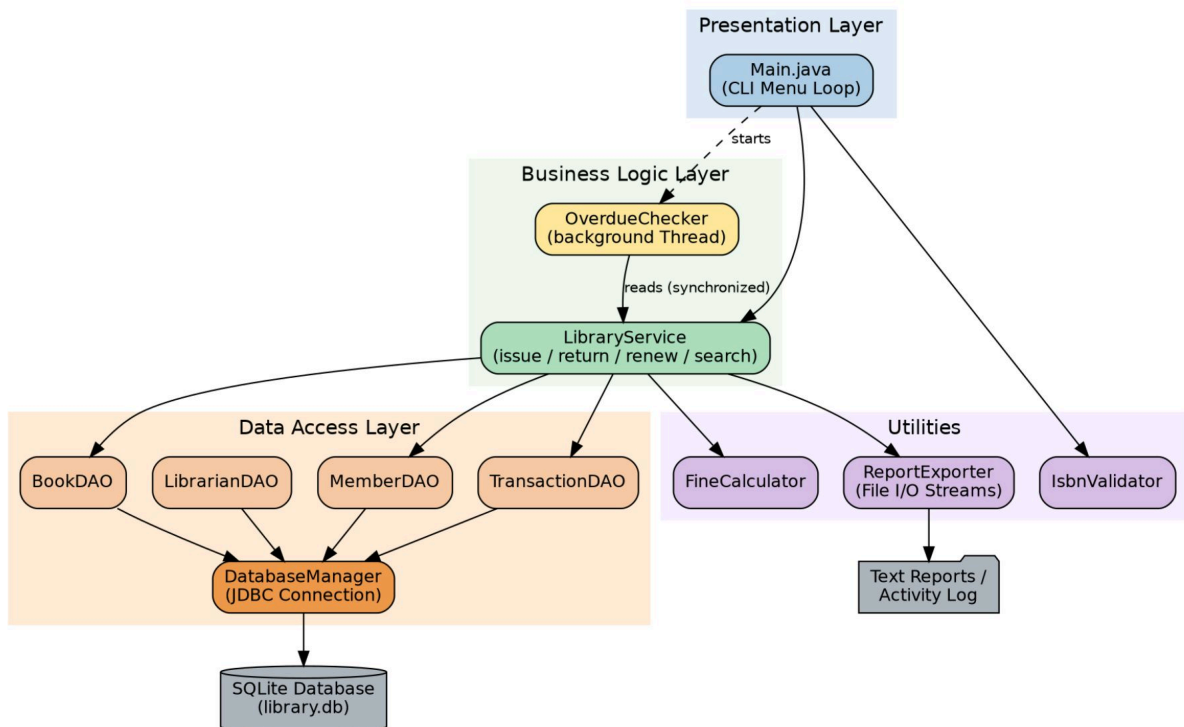


Figure 1. System architecture.

Layer Responsibilities

Presentation Layer

`ui.Main` displays the menu, reads input from the user, calls the service layer, and displays the results. It is also responsible for catching exceptions and converting them into messages that make sense to the user.

Business Logic Layer

`service.LibraryService` contains the main library operations. It checks book availability, member limits, renewal eligibility, and fine calculations.

`thread.OverdueChecker` runs in the background and checks for overdue loans.

Data Access Layer

The classes in the `dao` package convert Java objects to database rows and database rows back to Java objects.

The DAO classes do not contain the library's business rules.

Utility Layer

The classes in the `util` package provide smaller independent functions:

- Fine calculation
- ISBN validation

- Report generation

6. Design Diagrams

6.1 Use Case Diagram

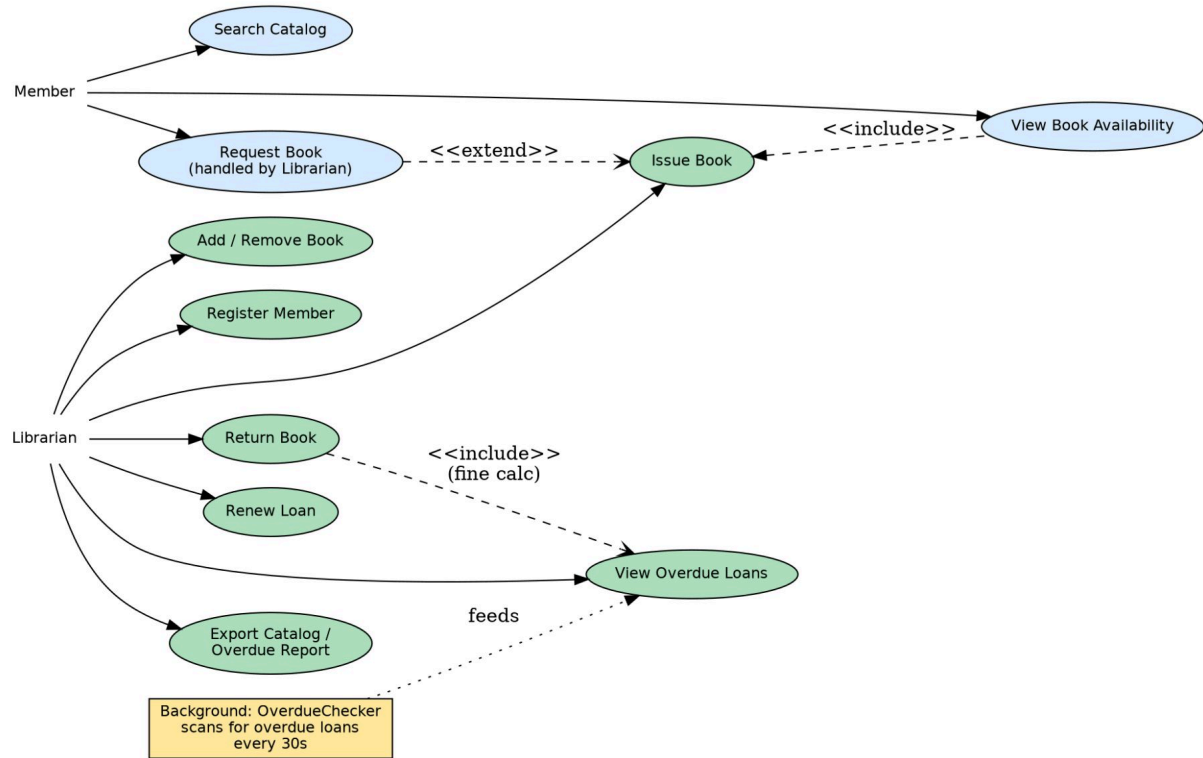


Figure 2. Use case diagram.

There are two actors in the system:

- Member
- Librarian

The member is currently represented indirectly because the application is operated by a librarian through a single command-line interface. The librarian performs the actions directly.

The diagram also includes the background OverdueChecker, which updates overdue tracking independently of a manual request.

6.2 Workflow Diagram: Issue Book

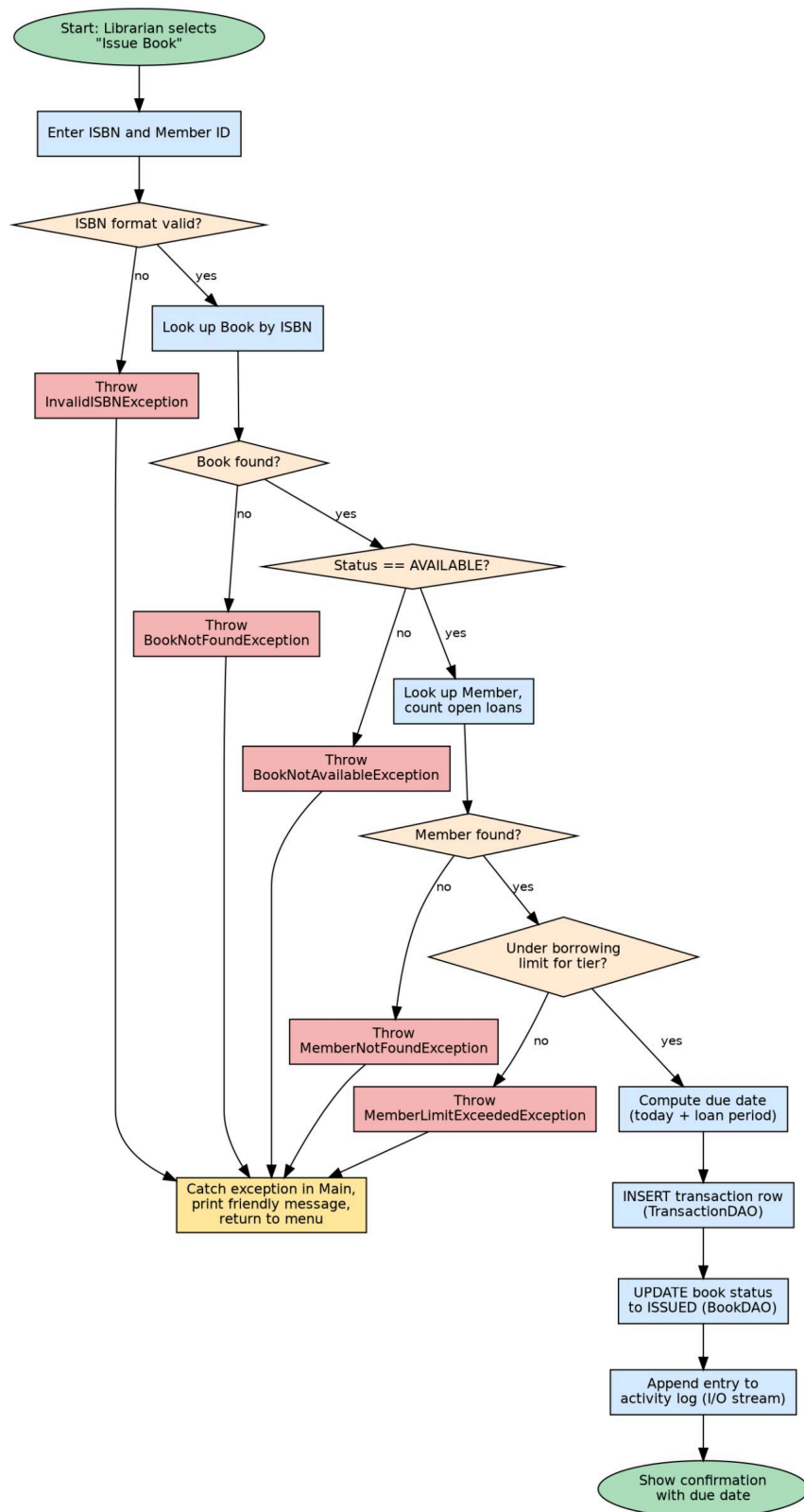
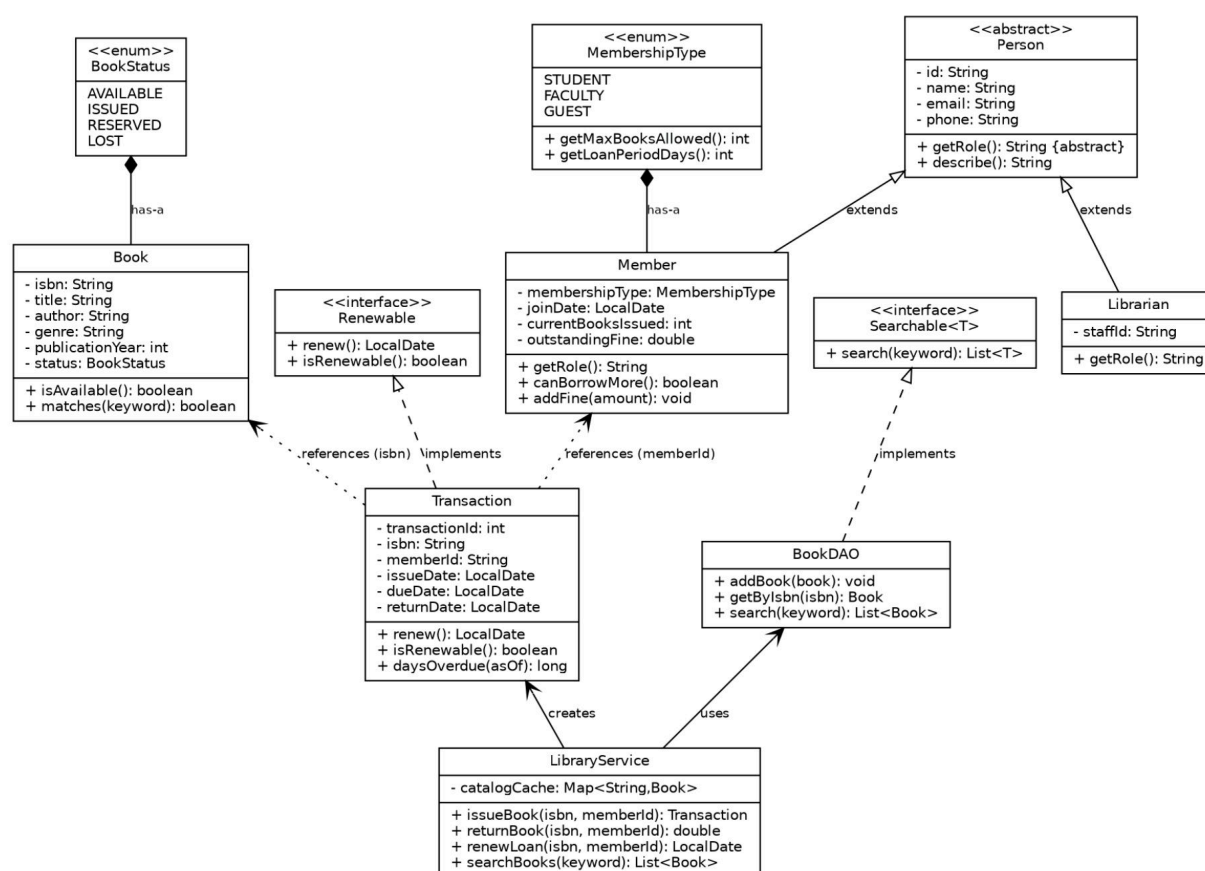


Figure 3. Workflow diagram for issuing a book.

This flowchart shows the validation order used by `LibraryService.issueBook()`:

Each major failure case is represented by a separate custom exception.



The main relationships are:

6.4 Sequence Diagram: Issue Book

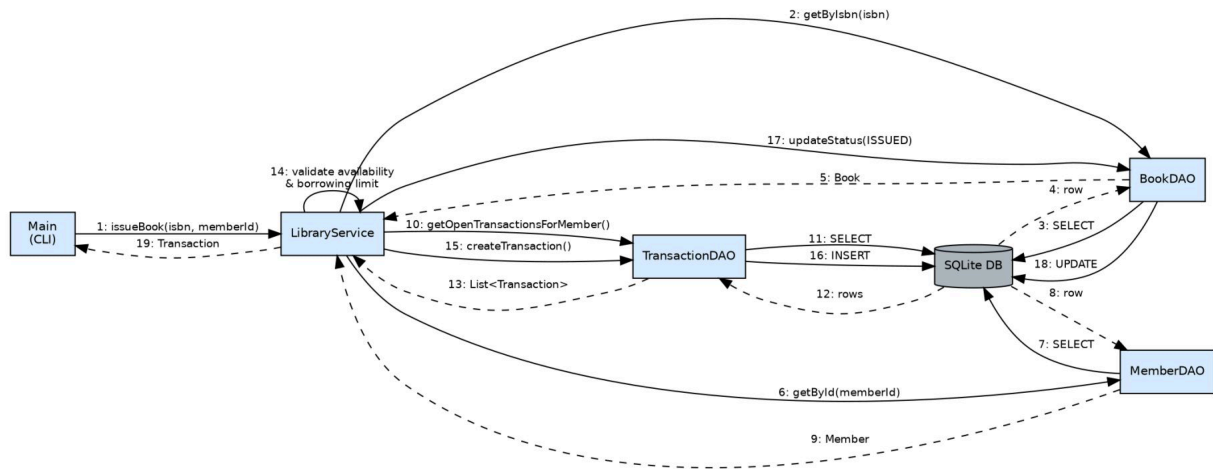


Figure 5. Sequence diagram for issuing a book.

The sequence diagram shows the call path for `issueBook()`:

- 1 The CLI receives the ISBN and member ID.
- 2 LibraryService validates the request.
- 3 BookDAO and MemberDAO are used to retrieve the required records.
- 4 TransactionDAO inserts the new loan.
- 5 BookDAO updates the book's status to Issued.

6.5 ER Diagram

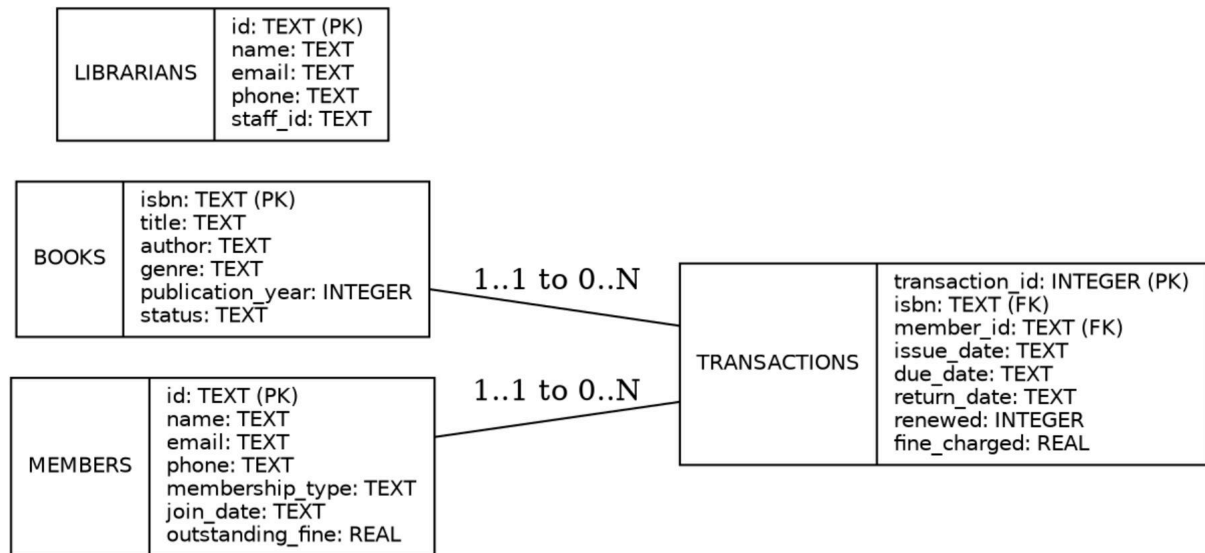


Figure 6. Entity-relationship diagram.

The database contains four main tables:

- books

- members
- librarians
- transactions

The transactions table refers to both books and members. A book can appear in multiple transaction records over time, and a member can also have multiple transaction records. Foreign keys are used to connect these tables.

7. Design Decisions and Rationale

SQLite

SQLite was chosen instead of a client-server database because the project needs persistent storage without requiring the evaluator to install and configure a separate database server.

SQLite works as a local file, while still allowing the application to use JDBC.

Abstract `Person` Class

Member and Librarian share several fields:

- ID
- Name
- Email
- Phone

They also have different roles and behaviour. The abstract Person class provides the shared fields and methods, while the subclasses implement role-specific behaviour through `getRole()`.

This gives the project a practical use of inheritance and runtime polymorphism.

Checked and Unchecked Exceptions

Business-rule problems are represented by checked exceptions that extend `LibraryException`. Examples include an unavailable book and a missing member.

`InvalidISBNException` is unchecked because ISBN validation is performed at the input boundary. Invalid input should be rejected immediately instead of being passed through every method signature.

`OverdueChecker` Thread

The project requirements include multithreading through the Thread class, so `OverdueChecker` extends `Thread`.

The thread is marked as a daemon thread. This means it does not prevent the application from shutting down after the librarian exits.

Synchronization

The methods `issueBook()`, `returnBook()`, and `renewLoan()` are synchronized on the shared `LibraryService` object.

The background thread may read loan information while the main thread is changing it. Synchronization prevents the main thread and background thread from updating related data at the same time.

`HashMap` Cache

LibraryService contains a small HashMap cache for frequently accessed book records.

The DAOs also use ArrayList objects where appropriate. The HashMap avoids an unnecessary database lookup when the same ISBN is requested repeatedly.

8. Implementation Details

The project uses Maven and targets Java 17.

The main dependencies are:

- org.xerial:sqlite-jdbc
- JUnit 5

The project contains 20 main-source files and 3 test files. The files are organised into six main packages.

Package	Contents
model	Person, Member, Librarian, Book, Transaction, BookStatus, MembershipType, Searchable<T>, and Renewable
exception	LibraryException, BookNotAvailableException, BookNotFoundException, MemberNotFoundException, MemberLimitExceededException, and related exception classes
dao	DatabaseManager, BookDAO, MemberDAO, LibrarianDAO, and TransactionDAO
service	LibraryService
thread	OverdueChecker
util	FineCalculator, IsbnValidator, and ReportExporter
ui	Main

JDBC Access

All database operations use PreparedStatement objects with ? placeholders.

The application does not build SQL statements by joining strings with user input. This includes the generated-key retrieval used when a new transaction is inserted.

Database Initialization

`DatabaseManager.initializeSchema()` uses `CREATE TABLE IF NOT EXISTS`.

This allows the application to start repeatedly using the same database file without needing a separate migration command.

Fine Calculation

The fine calculation is kept in `FineCalculator`, which is a stateless utility class.

The project uses the following rule:

- There is a one-day grace period.
- After the grace period, the fine is \$5 per day.
- The fine cannot become negative.

Keeping the formula in one class makes it easier to test and change.

Reports and Logging

`ReportExporter` writes reports using `BufferedWriter` and `FileWriter`.

The report files and activity log use character-stream I/O with `try-with-resources`. No external logging library is used.

9. Screenshots and Results

The following sample session shows two books being added, a member being registered, a book being issued and returned, and a catalog report being exported.

```

=====
LIBRARY MANAGEMENT SYSTEM
=====

----- MAIN MENU -----
1. Add Book
2. View All Books
...
0. Exit
Enter your choice: 1
ISBN (10 or 13 digits): 9780132350884
Title: Clean Code
Author: Robert C. Martin
Genre: Software Engineering
Publication year: 2008
Book added successfully.

Enter your choice: 2
9780132350884 | Clean Code | Robert C. Martin | Software Engineering | 2008 | Available
9780134685991 | Effective Java | Joshua Bloch | Programming | 2018 | Available

Enter your choice: 7
ISBN: 9780132350884
Member ID: M001
Book issued. Txn#1 | ISBN:9780132350884 | Member:M001 | Issued:2026-09-12 |
Due:2026-09-26 | OPEN

Enter your choice: 8
ISBN: 9780132350884
Member ID: M001
Book returned on time. No fine.

Enter your choice: 12
Catalog report written to: reports/catalog_2026-09-12_04-09-11.txt

Enter your choice: 0
Shutting down background threads...
Goodbye!

```

The corresponding exported catalog report is:

```

LIBRARY CATALOG REPORT
Generated: 2026-09-12T04:09:11.190285710
=====
9780132350884 | Clean Code | Robert C. Martin | Software Engineering | 2008 | Available
9780134685991 | Effective Java | Joshua Bloch | Programming | 2018 | Available
=====
Total books: 2

```

The activity log contains entries similar to the following:

```

[2026-09-12T04:09:11.177789589] ISSUED book 9780132350884 to member M001, due
2026-09-26
[2026-09-12T04:09:11.188307314] RETURNED book 9780132350884 from member M001, on time
[2026-09-12T04:09:11.197121447] OverdueChecker thread stopped.

```

10. Testing Approach

Testing was carried out at two levels:

- 1 Unit testing with JUnit 5
- 2 Manual functional testing through the command-line application

Unit Tests

The unit tests focus on logic that does not require a database.

``FineCalculatorTest``

This test class checks:

- No fine during the grace period
- Correct daily fine after the grace period
- The fine never becomes negative

``IsbnValidatorTest``

This test class checks that the validator:

- Accepts valid 10-digit ISBNs
- Accepts valid 13-digit ISBNs
- Accepts ISBNs containing hyphens
- Rejects incorrect lengths
- Rejects non-numeric input

``TransactionRenewalTest``

This test class checks that:

- An open loan that is not yet due can be renewed
- A loan can be renewed only once
- An overdue loan cannot be renewed
- Attempting to renew an overdue loan throws `IllegalStateException`

The logic was also checked through direct execution. The final result was 14 out of 14 assertions passing.

Manual Functional Testing

The full CLI was tested with a real SQLite database.

The manual tests included:

- Adding books
- Adding members

- Issuing an available book
- Trying to issue an already-issued book
- Issuing a book to a member who had reached the borrowing limit
- Entering a malformed ISBN
- Returning a book on time
- Returning a book after the due date
- Checking the fine calculation
- Renewing a loan
- Exporting the catalog report
- Exporting the overdue report

The negative cases produced the expected custom exceptions and displayed readable error messages instead of stack traces.

11. Challenges Faced

Tracking Currently Issued Books

The system does not store a separate running loan count in the member's table.

Instead, it calculates the count from open transaction records. This avoids having two separate values that could become inconsistent.

The trade-off is that the application performs an additional query when it needs the current count. This is acceptable for the size of this project.

Making the Background Thread Safe

The `OverdueChecker` thread reads loan information while the main thread may issue or return a book.

Without synchronization, two operations could access or update related records at the same time.

The issue, return, and renewal methods were therefore synchronized on the shared `LibraryService` instance.

Designing the Exception Hierarchy

The first version of the application caught most failures as a generic `Exception`.

That made it difficult to display useful messages because different problems looked the same to the user.

The final version separates business-rule problems into the checked `LibraryException` family and keeps ISBN input validation in the unchecked `InvalidISBNException`.

12. Learnings and Key Takeaways

Several lessons came from developing the project.

- A small exception hierarchy made the command-line error handling easier to understand.
- Separating business exceptions from invalid input made the method signatures clearer.
- Adding a background thread required checking which methods could run concurrently.
- Synchronization was needed for the operations that could modify the same loan and book data.
- Keeping SQL inside the DAO classes made the service layer easier to inspect.
- Having one service class for business rules made changes to borrowing limits, renewal rules, and fines easier to manage.
- The HashMap cache showed how a collection can be used for a specific performance reason rather than only to satisfy a syllabus requirement.

13. Future Enhancements

Possible future improvements include:

- Adding a reservation queue for books that are currently issued
- Adding member login and authentication
- Allowing members to view their own loans and fines
- Replacing some JDBC boilerplate with JPA or Hibernate
- Adding a REST API around LibraryService
- Reusing the same business logic for a future web or mobile application

14. References

- Herbert Schildt, Java: The Complete Reference, 11th Edition, Oracle Press, 2018.
- Cay S. Horstmann, Core Java Volume I - Fundamentals, 11th Edition, Pearson, 2018.
- Oracle Java SE 17 Documentation: <https://docs.oracle.com/en/java/javase/17/>
- Oracle, Java Platform, Standard Edition, The Java Tutorials, JDBC Basics: <https://docs.oracle.com/javase/tutorial/jdbc/>
- SQLite JDBC Driver, xerial/sqlite-jdbc: <https://github.com/xerial/sqlite-jdbc>
- SQLite Documentation: <https://sqlite.org/docs.html>
- Oracle, Java Concurrency and Multithreading: <https://docs.oracle.com/javase/tutorial/essential/concurrency/>
- Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1994.
- JUnit 5 User Guide: <https://junit.org/junit5/docs/current/user-guide/>
- Apache Maven Documentation: <https://maven.apache.org/guides/>